CAI104 Concepts in AI
Assessment 3: Robot Path Planner

Student: _____

1. Problem Formulation

The task is to plan an optimal (shortest) path for a robot that must travel from its starting cell to a target cell inside a 12 x 24 grid maze (288 cells). The maze is given as a 2D array using the legend: 1 = wall, 0 = walkable space, -1 = robot start, 9 = target. The robot may move one cell at a time in four directions (up, down, left, right); diagonal moves are not allowed and every move has a uniform cost of 1.

Formulated as a state-space search problem:

- **State:** each walkable cell (row, col).
- **Initial state:** the cell containing -1.
- **Goal test:** the cell containing 9.
- **Actions / successor function:** move to an orthogonally adjacent cell that is inside the grid and is not a wall.
- **Step cost:** 1 per move; the path cost is the number of moves.

Because every step costs the same and we want the fewest-move route, the problem is a single-source shortest-path search on an unweighted graph whose nodes are cells and whose edges connect adjacent walkable cells.

2. Choice of Algorithm: A*

I selected A* (A-star) search. A* is an informed search algorithm that always expands the frontier node minimising f(n) = g(n) + h(n), where g(n) is the real cost from the start to n and h(n) is a heuristic estimate of the remaining cost from n to the goal. For a 4-connected grid the Manhattan distance |dr| + |dc| is the natural heuristic: it never overestimates the true distance (it is admissible) and it is consistent, which together guarantee that A* returns the optimal path.

A* was chosen over the alternatives because:

- **vs BFS:** Breadth-First Search would also find the optimal unweighted path but expands cells blindly in all directions, exploring far more of the maze.
- **vs DFS:** Depth-First Search is memory-light but does not guarantee the shortest path and can wander into long dead-ends.

- **vs Greedy:** Greedy Best-First Search expands fewer nodes but ignores the cost already paid, so it can return a sub-optimal route.

A* keeps the optimality of BFS while using the heuristic to steer the search toward the goal, making it both correct and efficient — the ideal fit for robot navigation.

3. Pseudocode

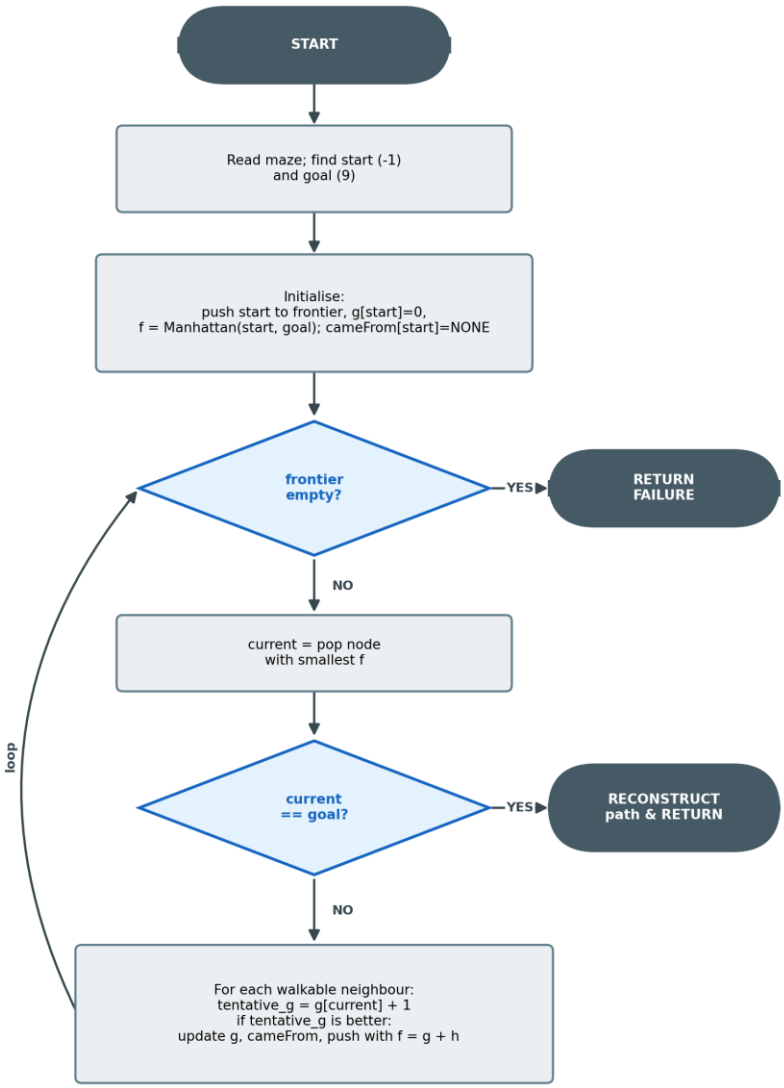
(Comments begin with //. This pseudocode is not part of the word count.)

```
ALGORITHM A_Star_PathPlanner(maze)

  // --- Setup ---
  start  <- cell where maze = -1          // robot initial position
  goal   <- cell where maze =  9          // target
  MOVES  <- {(-1,0),(1,0),(0,-1),(0,1)}   // up, down, left, right

  frontier <- empty min-priority-queue ordered by f = g + h
  gScore   <- map with gScore[start] = 0  // best cost start -> cell
  cameFrom <- map with cameFrom[start] = NONE
  PUSH(frontier, start, priority = Manhattan(start, goal))

  // --- Main search loop ---
  WHILE frontier is not empty DO
      current <- POP node with smallest f from frontier

      IF current = goal THEN                 // goal reached
          RETURN Reconstruct(cameFrom, goal)

      FOR each move IN MOVES DO
          next <- current + move
          IF next is inside maze AND maze[next] != WALL THEN
              tentative_g <- gScore[current] + 1     // each step costs 1
              IF next not in gScore OR tentative_g < gScore[next] THEN
                  gScore[next]   <- tentative_g       // a cheaper route
                  cameFrom[next] <- current
                  f <- tentative_g + Manhattan(next, goal)
                  PUSH(frontier, next, priority = f)
              END IF
          END IF
      END FOR
  END WHILE

  RETURN FAILURE          // frontier emptied -> goal unreachable


FUNCTION Manhattan(a, b)                     // admissible heuristic
  RETURN |a.row - b.row| + |a.col - b.col|
```

```
FUNCTION Reconstruct(cameFrom, goal)          // build path start -> goal
  path <- empty list
  node <- goal
  WHILE node != NONE DO
      add node to front of path
      node <- cameFrom[node]
  RETURN path
```

4. Flowchart

The control flow of the A* path planner:

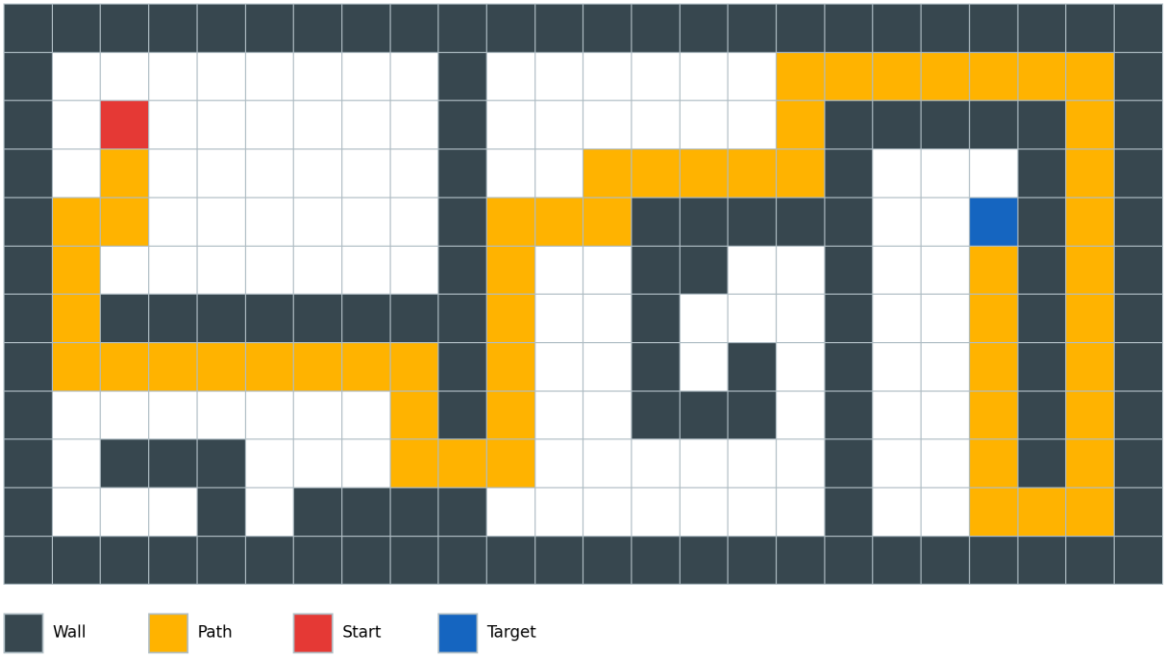


5. Implementation and Result

The algorithm was implemented in Python 3 (see path_planner.py in the Source folder). Running it on the supplied maze produces:

- Start = (row 2, col 2); Target = (row 4, col 20).
- A* expanded 148 nodes before reaching the goal.
- Optimal path length = 55 cells (54 moves).

The target sits in a small pocket that is walled off from the direct approach, so the optimal route loops down and around the maze. The program marks the path and prints the solved grid; the rendered result is shown below.



Reflective Report

Overview

The objective of this project was to design and implement an artificial intelligence agent capable of planning an optimal path for a robot navigating a two-dimensional maze. The environment was supplied as a 12 by 24 grid of 288 cells, encoded as a 2D array in which 1 represents a wall, 0 a free space, -1 the robot's starting position and 9 the target. My task was to choose an appropriate AI search technique, express it clearly as pseudocode, implement it in a modern programming language, and reflect critically on the development experience. After weighing the trade-offs between uninformed and informed search, I selected the A* algorithm with a Manhattan-distance heuristic, implemented in Python 3, and validated it against the maze provided in the assessment brief.

My approach moved deliberately from theory to practice. I began by formulating the maze as a classical state-space search problem, identifying the states, the initial state, the goal test, the successor function and the step cost. Framing the problem in these familiar terms made it straightforward to map the textbook description of A* onto the concrete grid. I then wrote the pseudocode, translated it line by line into Python, and tested the result, iterating until the program returned a verifiably shortest path. The final solution finds the optimal 54-move route from the start at (2, 2) to the target at (4, 20), expanding only 148 of the 288 cells in the process, and visualises the path directly on the grid so the result can be checked by eye.

My working environment was deliberately lightweight. I used Python 3 with only the standard library, relying on the built-in heapq module for the priority queue so that the solution has no external dependencies and can be run by anyone with a basic Python install. I kept a single, well-commented source file and tested it incrementally after each change, which suited a project of this size and meant I always had a runnable program to fall back on. This overview sets the scene for the reflection that follows, in which I examine what worked well, what caused difficulty, where my confidence is still incomplete, and what I take away from the exercise as a whole.

What Went Right

The most satisfying aspect of the project was that the choice of algorithm proved to be the right one and paid off exactly as the theory predicted. A* with the Manhattan-distance heuristic returned a provably optimal path on the first complete run, and because the heuristic is admissible and consistent on a four-connected grid, I could be confident the result was genuinely the shortest route rather than merely a working one. Seeing the rendered grid trace a clean corridor from start to target was strong evidence that the successor function, the cost model and the priority ordering were all behaving correctly together.

Writing the pseudocode before any code was a decision that repaid itself many times over. By the time I started typing Python, the structure of the algorithm — the priority queue keyed on f = g + h, the gScore and cameFrom maps, and the path reconstruction step — was already

settled, so the implementation became a faithful transcription rather than a struggle. The use of Python's heapq module for the priority queue kept the frontier management both efficient and concise, and decomposing the program into small, single-purpose functions (find_cell, manhattan, neighbours, a_star, reconstruct and mark_and_print) made each part easy to reason about and test in isolation.

Testing also went smoothly because I built the visualisation early. Rather than inspecting raw coordinate lists, I had the program render the solved maze, which made bugs immediately visible: a path that clipped a wall or skipped a cell would have stood out at a glance. This quick feedback loop turned debugging from guesswork into observation. Finally, instrumenting the search to count the number of expanded nodes gave me a tangible efficiency measure — 148 expansions — which let me confirm that A* was indeed exploring far less of the space than an uninformed search would have, validating the core reason I chose it.

On a personal level, the project deepened my understanding of why the distinction between informed and uninformed search matters in practice rather than just on paper. Before this assessment I could recite the definition of an admissible heuristic, but implementing one and then watching the expansion count fall confirmed the idea in a way that reading never had. I also gained a clearer appreciation of how the components of A* depend on one another: the heuristic only guarantees optimality because the step costs are uniform and the goal test is applied on expansion rather than on generation, and getting all three to agree is what made the result trustworthy.

What Went Wrong

The development was not without difficulty. My first instinct was to reach for a simpler Breadth-First Search, since it also guarantees the shortest path on an unweighted grid. I implemented it, and it worked, but profiling showed it expanded substantially more cells than necessary because it fanned out uniformly in every direction. This pushed me to commit properly to A*, which meant introducing a priority queue and a heuristic — more moving parts, and therefore more opportunities for mistakes.

The first of those mistakes was a stale-entry problem in the priority queue. Because Python's heapq does not support decreasing the priority of an element already in the heap, I push a new entry whenever a cheaper route to a cell is found, which leaves outdated entries behind. My initial version did not account for this and occasionally processed a cell using an out-of-date cost. I resolved it by storing the authoritative cost in the gScore map and skipping any popped entry whose cost is worse than the recorded best, a standard and robust guard. A second, more embarrassing error was an off-by-one in my neighbour generation: an early version allowed the search to step outside the grid boundary, which raised an index error. Adding an explicit bounds check inside the neighbours function fixed it and reminded me to treat the grid edges as carefully as the walls.

I also briefly misread the maze itself. The target at (4, 20) is tucked inside a pocket that is almost entirely enclosed by walls, so the optimal path is a long, looping route rather than the short hop

the coordinates might suggest. For a short while I suspected my code was wrong, when in fact it was correctly reporting the only legal way in. The lesson was to trust a verified algorithm and to study the environment carefully before doubting the implementation.

With hindsight, a recurring theme in these problems was that each one stemmed from an assumption I had not made explicit: that the heap would stay consistent on its own, that neighbours would always be valid cells, and that a nearby goal would have a short path. Making those assumptions visible — by guarding them in code and by inspecting the maze carefully — is what ultimately resolved each bug. If I were to start again I would write a few small unit tests for the helper functions up front, rather than relying solely on the end-to-end visualisation, so that errors of this kind surfaced even sooner.

What I Am Not Sure About

Several aspects of the solution leave me with open questions. The first concerns the handling of ties. When two cells share the same f value, my implementation breaks the tie by the natural ordering of the queue entries, which is effectively arbitrary. This does not affect the length of the optimal path — A* still returns a shortest route — but it can change which of several equally short paths is returned. I am not certain whether a deliberate tie-breaking rule, such as preferring the node with the larger g value, would produce a more natural-looking or more consistent path, and this would be worth investigating.

I am also unsure how well the current design would scale or generalise. On a 288-cell grid performance is irrelevant, but the brief notes that the facilitator may supply a much larger or randomly generated maze. For very large grids a more memory-conscious variant such as Iterative Deepening A* might be preferable, and I have not tested where my implementation's practical limits lie. Similarly, my cost model assumes uniform step costs and four-directional movement; if the robot were allowed diagonal moves, the Manhattan heuristic would no longer be admissible and I would need to switch to an octile or Euclidean heuristic. I am confident about why this is true in theory but would want to verify it empirically before relying on it.

A further uncertainty is how my confidence in the result should be balanced against the limits of testing. I validated the planner against the single maze supplied in the brief and a couple of hand-made variations, and it behaved correctly each time, but a handful of successful runs is not a proof of robustness. I am not entirely sure the implementation would cope gracefully with degenerate inputs such as a maze with no path to the goal, a start cell equal to the goal cell, or a malformed grid. I added a failure return for the unreachable case and reasoned through these situations, but I would want a systematic test suite covering them before I would describe the program as production-ready rather than fit for this assessment.

Finally, I am uncertain about the ethical and safety dimensions of deploying such a planner in the real world. A grid abstraction assumes perfect knowledge of the environment and flawless sensing, whereas a real robot faces noisy sensors, moving obstacles and the risk of harm if it miscalculates. How an optimal-on-paper path should be tempered by safety margins,

uncertainty and real-time replanning is something I understand only at a surface level and would need to study further.

Conclusion

This project gave me a concrete, end-to-end experience of solving a simplified real-world problem with an AI search technique. By formulating the maze as a state-space problem, selecting A* on principled grounds, designing it as careful pseudocode and implementing it in Python, I produced an agent that reliably finds the optimal path and clearly visualises it. The work reinforced why informed search is so widely used in robotics and navigation: a good admissible heuristic delivers the correctness of exhaustive search at a fraction of the exploration cost, as my node-expansion measurements confirmed.

Just as valuable were the difficulties. Wrestling with stale priority queue entries, boundary conditions and a deceptively enclosed target taught me that the gap between a textbook algorithm and a robust implementation is filled with exactly these small, practical details. The questions I am still unsure about — tie-breaking, scalability, heuristic choice under different movement models, and the ethics of real-world deployment — point clearly to where my understanding can deepen next. Overall the assessment achieved its aim: it connected the theory of uninformed and informed search to a working program and left me with both a functioning path planner and a much sharper sense of how AI problem-solving methods behave in practice.

— End of Report —